
On Maximizing the Net Present Value of a Project under Renewable Resource Constraints

Author(s): Mario Vanhoucke, Erik Demeulemeester and Willy Herroelen

Source: *Management Science*, Aug., 2001, Vol. 47, No. 8 (Aug., 2001), pp. 1113-1121

Published by: INFORMS

Stable URL: <https://www.jstor.org/stable/822597>

REFERENCES

Linked references are available on JSTOR for this article:

https://www.jstor.org/stable/822597?seq=1&cid=pdf-reference#references_tab_contents

You may need to log in to JSTOR to access the linked references.

JSTOR is a not-for-profit service that helps scholars, researchers, and students discover, use, and build upon a wide range of content in a trusted digital archive. We use information technology and tools to increase productivity and facilitate new forms of scholarship. For more information about JSTOR, please contact support@jstor.org.

Your use of the JSTOR archive indicates your acceptance of the Terms & Conditions of Use, available at <https://about.jstor.org/terms>



JSTOR

INFORMS is collaborating with JSTOR to digitize, preserve and extend access to *Management Science*

On Maximizing the Net Present Value of a Project Under Renewable Resource Constraints

Mario Vanhoucke • Erik Demeulemeester • Willy Herroelen

Operations Management Group, Department of Applied Economics, Katholieke Universiteit Leuven,
Naamsestraat 69, B-3000 Leuven, Belgium

mario.vanhoucke@econ.kuleuven.ac.be • erik.demeulemeester@econ.kuleuven.ac.be
willy.herroelen@econ.kuleuven.ac.be

In this paper we study the resource-constrained project-scheduling problem with discounted cash flows. Each activity of this resource-constrained project-scheduling problem has certain resource requirements and a known deterministic cash flow that can be either positive or negative. Deterministic cash flows are assumed to occur over the duration of the activities. Progress payments and cash outflows occur at the completion of activities. The objective is to schedule the activities subject to a fixed deadline to maximize the net present value subject to the precedence and resource constraints. With these features the financial aspects of project management are taken into account.

We introduce a depth-first branch-and-bound algorithm that makes use of extra precedence relations to resolve a number of resource conflicts and a fast recursive search algorithm for the max-*npv* problem to compute upper bounds. The recursive search algorithm exploits the idea that positive cash flows should be scheduled as early as possible while negative cash flows should be scheduled as late as possible within the precedence constraints. The procedure has been coded in Visual C++, Version 4.0 under Windows NT, and has been validated on two problem sets.

(Resource-Constrained Project Scheduling; Discounted Cash Flows; Branch-And-Bound)

1. Introduction

In recent years a number of publications have dealt with the project-scheduling problem under the objective of maximizing the net present value (*npv*) of the project. The majority of the contributions assume a completely deterministic project setting in which all relevant problem data, including the various cash flows, are assumed known from the outset. Research efforts have led to exact procedures for the unconstrained project-scheduling problem, in which activities are only subject to precedence constraints. In addition, numerous efforts aim at providing optimal or suboptimal solutions to the project-scheduling problem under various types of resource constraints,

using a rich variety of assumptions with respect to network representation (activity-on-the-node versus activity-on-the-arc), cash flows patterns (positive and/or negative, event-oriented or activity-based), and resource constraints (capital constrained, different resource types, materials considerations, time/cost trade-offs). A number of efforts focus on the simultaneous determination of both the amount and timing of payments. Last, a modest start has been made in tackling the stochastic aspects of the scheduling problem involved. For a recent review of the vast literature and a categorization of the solution procedures, we refer the reader of Özdamar and Ulusoy (1995) and Herroelen et al. (1997).

In this paper we present a branch-and-bound algorithm to maximize the net present value in project networks with zero-lag finish-start precedence constraints and renewable resource constraints (problem $m, 1|cpm, \delta_n, c_j|npv$ following the classification scheme of Herroelen et al. 1999 and further denoted as *RCPSPDC*, i.e., the resource-constrained project-scheduling problem with discounted cash flows). The proposed solution procedure makes use of a fast recursive search algorithm for the unconstrained max-*npv* project-scheduling problem (problem $cpm, \delta_n, c_j|npv$, further denoted as the max-*npv* problem) to compute upper bounds.

The branch-and-bound procedure makes use of the concept of minimal delaying alternatives that was introduced by Demeulemeester and Herroelen (1992, 1997) and additional precedence relations to resolve resource conflicts due to Icmeli and Erengüç (1996). For an overview of the existing algorithms in the literature for both the max-*npv* problem and the *RCPSPDC*, we refer to Vanhoucke et al. (1999). The organization of the paper is as follows. In §2 we discuss the max-*npv* problem. Section 3 reviews the recursive search algorithm for the max-*npv* problem. In §4 we discuss the *RCPSPDC*. Section 5 describes a branch-and-bound approach for this *RCPSPDC*. In §6 we report detailed numerical results on two problem sets. Section 7 contains overall conclusions.

2. The Deterministic Max-*npv* Problem

The deterministic max-*npv* problem involves the scheduling of project activities to maximize the net present value of the project in the absence of resource constraints. The project is represented by an activity-on-the-node (AoN) network $G = (N, A)$, where the set of nodes, N , represents activities, and the set of arcs, A , represents finish-start precedence constraints with a time lag of zero. The project must finish before its imposed deadline δ_n . The activities are numbered from the dummy start activity 1 to the dummy end activity n . The duration of an activity is denoted by d_i ($1 \leq i \leq n$) and the performance of each activity involves a series of cash flow

payments and receipts throughout the activity duration. Assume that cf_{it} ($1 < i < n$) denotes the known deterministic cash flow of activity i in period t of its execution. A terminal value of each activity on completion can be calculated by compounding the associated cash flow to the end of the activity as follows: $c_i = \sum_{t=1}^{d_i} cf_{it} e^{\alpha(d_i-t)}$, where α represents the discount rate and c_i the terminal value of cash flows of activity i at its completion. If the nonnegative integer variable f_i ($1 \leq i \leq n$) denotes the completion time of activity i , its discounted value at the beginning of the project is $c_i e^{-\alpha f_i}$. A formulation of the max-*npv* problem can be given as follows:

$$\text{Maximize } \sum_{i=2}^{n-1} c_i e^{-\alpha f_i} \quad (1)$$

Subject to

$$f_i \leq f_j - d_j \quad \forall (i, j) \in A \quad (2)$$

$$f_n \leq \delta_n \quad (3)$$

$$f_1 = 0. \quad (4)$$

The exact solution procedures for solving the max-*npv* problem offered in the literature take an event-oriented view with net cash flows associated with the events in activity-on-the-arc (AoA) networks. Russell (1970) was the first to offer a nonlinear programming formulation and an exact solution procedure based on an iterative series of linear approximations and the solution of a transshipment problem over a network model. Apart from an example illustrating the application of the algorithm, he does not report any computational experience. Grinold (1972) also takes an event-oriented view and shows that the problem can be transformed into an equivalent linear program. Again, apart from an example illustrating the computations, no further computational results are given. Elmaghraby and Herroelen (1990) have developed an exact solution procedure for AoA networks based on the intuitive argument that positive cash flows should be scheduled as early as possible and negative cash flows should be scheduled as late as possible within the precedence constraints. The algorithm operates by building tree structures in an iterative fashion and by determining proper displacement intervals for the trees. Herroelen and Gallens (1993) have streamlined

the algorithm and report on favorable computational results on 250 randomly generated projects with a computer code written in the C language and running under the DOS operating system.

We would like to stress here that the specification of a project deadline δ_n is a definite requirement for any solution procedure for this problem, as the introduction of positive as well as negative cash flows might induce the project to be split into two parts. In practice, the deadline for project completion will be negotiated in advance with the customer. However, when a negotiated deadline does not exist, a very large project deadline can be set. This remark is of course equally valid for the resource-constrained version of the max-*npv* problem, which is described in §4.

3. An Exact Recursive Search Procedure for the Max-*npv* Problem

In this section we discuss the exact recursive search procedure for the max-*npv* problem as formulated in Equations (1)–(4). This algorithm will be used by the branch-and-bound procedure of §5 to compute upper bounds on the net present value of a resource-constrained project-scheduling problem with discounted cash flows where the activities are also subject to renewable resource constraints.

The proposed recursive algorithm (Demeulemeester et al. 1996) consists of three steps: the construction of an early tree, the construction of a current tree, and a recursive search on the current tree.

3.1. Step 1: Constructing the Early Tree

The algorithm starts in Step 1 by computing the earliest completion time for the activities based on traditional forward-pass critical-path calculations and determines the corresponding *early tree*, *ET*.

3.2. Step 2: Constructing the Current Tree

The early tree spans all activities scheduled at their earliest completion times and corresponds to a feasible solution with a project duration equal to the critical-path length. In Step 2 the algorithm builds the *current tree*, *CT*, by delaying, in reverse order, all

activities *i* with a negative cash flow and no successor in the early tree as much as possible within the early tree, i.e., by linking them to their earliest starting successor.

3.3. Step 3: Searching the Current Tree

In Step 3 the current tree is the subject of a *recursive search* (starting from the dummy start activity 1) to identify sets of activities (*SA*) that might be shifted forward (away from time zero) to increase the net present value of the project. Due to the structure of the recursive search, it can never happen that a backward shift (toward time zero) of a set of activities can lead to an increase in the *npv* of the project.

When a set of activities *SA* is found for which a forward shift leads to an increase of the *npv*, the algorithm computes its displacement interval and updates the current tree *CT* as follows. The arc (*i*, *j*) that connects a node $i \notin SA$ to a node $j \in SA$ in the current tree *CT* is removed from it. The displacement interval of the set of activities *SA* under consideration is computed as follows. Find a new arc (k^* , l^*) for which $v_{k^*l^*} = \min_{(k,l) \in A, k \in SA, l \notin SA} \{f_l - d_l - f_k\}$ and add it to the current tree. The completion times of the activities of *SA* are then increased by the minimal displacement $v_{k^*l^*}$, and the algorithm repeats the recursive search. If no further shift can be accomplished, the algorithm stops, and the completion times of the activities of the project with its corresponding *npv* are reported.

4. The Deterministic RCPSPDC

The RCPSPDC involves the scheduling of the project activities to maximize the net present value subject to precedence and resource constraints and differs only from the max-*npv* problem in the consideration of the use of resources for the different activities. There are *K* renewable resources with constant availability a_k ($1 \leq k \leq K$). Let r_{ik} ($1 < i < n, 1 \leq k \leq K$) denote the resource requirements of activity *i* with respect to resource type *k*. A conceptual formulation of the resource constraints can be given as follows:

$$\sum_{i \in S(t)} r_{ik} \leq a_k \quad k = 1, 2, \dots, K, t = 1, 2, \dots, \delta_n, \quad (5)$$

where $S(t)$ denotes the set of activities in progress in period $[t - 1, t]$.

Among the exact algorithms for solving the *RCPSPDC* (Herroelen et al. 1997), the branch-and-bound procedure of Icmeli and Erengüç (1996) is reported to be the most efficient. The algorithm extends the resource-constrained project-scheduling algorithm of Demeulemeester and Herroelen (1992) by adapting the branching strategy to cope with the *max-npv* criterion. Bounds are computed by solving unconstrained *max-npv* problems using the procedure of Grinold (1972). The authors have tested their algorithm on 50 Patterson problems (Patterson 1984) and 40 problems generated by *ProGen* (Kolisch et al. 1995) and have shown that it convincingly outperforms the exact procedure of Yang et al. (1992). In the next section we describe a depth-first branch-and-bound algorithm, which we feel outperforms the Icmeli and Erengüç procedure as well.

5. The Branch-and-Bound Algorithm

5.1. Description of the Search Tree and Branching Strategy

At the initial node of the branch-and-bound tree, we relax the problem by dropping the resource constraints of Equation (5) and exploit the fact that the optimal solution to the resulting unconstrained *max-npv* problem yields an upper bound *ub*. If the solution obtained by the recursive search procedure described in §3 is resource feasible, we have found the optimal solution for the *RCPSPDC* and the procedure terminates. If, however, the solution shows a resource conflict, we branch into the next level of the branch-and-bound tree to generate a set of delaying alternatives. A resource conflict occurs when there is at least one period $]t - 1, t]$ for which $\exists k \leq K: \sum_{i \in S(t)} r_{ik} > a_k$. A delaying alternative is a set of activities that, if delayed, resolves the resource conflict. According to Demeulemeester and Herroelen (1992, 1997), only minimal delaying alternatives must be considered to resolve a resource conflict. A delaying set *DS*, containing one or more delaying alternatives, can give rise to several minimal delaying modes (De Reyck and Herroelen 1998). For each delaying mode of the minimal delaying set *DS*, we append

additional precedence relations to resolve the resource conflict (Icmeli and Erengüç 1996) and compute a new upper bound using the recursive search procedure of §3. If, for example, Activities 1, 2, and 3 cause a resource conflict and the delaying set contains two minimal delaying alternatives, i.e., $DS = \{\{1, 2\}, \{3\}\}$, then the three different delaying modes are $(1 < 3)$, $(2 < 3)$, and $(3 < 1, 2)$. Each delaying mode corresponds to a node in the branch-and-bound tree that will be further explored during the branching process. We select among these nodes the delaying mode with the largest *ub*. If, on the one hand, the upper bound of a node corresponds to a solution that is resource feasible, we update the lower bound *lb* of the project (initially set to $-\infty$) and explore the next delaying mode at this level. If, on the other hand, the upper bound of a node is lower than or equal to the current lower bound, we fathom this node and backtrack to the previous level of the search tree. If the upper bound is greater than the current lower bound, we generate a new set of delaying alternatives at the next level of the tree. If there are no delaying modes left unexplored, we backtrack and proceed in the same way at the previous level. The algorithm stops when we backtrack to the initial level of the branch-and-bound tree.

5.2. Node-Fathoming Rules

Essentially, each node in the branch-and-bound tree represents the initial project network extended with a set of zero-lag finish-start precedence constraints to resolve resource conflicts. Therefore, the *subset dominance rule* developed by De Reyck and Herroelen (1998) (further denoted as the *DR&H* subset dominance rule) for the resource-constrained project-scheduling problem with generalized precedence relations also holds for the *RCPSPDC* and can be applied when a node *x* is compared with a previously examined node *y* on *another path* in the branch-and-bound tree. If the set of added precedence constraints in node *x* contains as a subset another set of precedence constraints at node *y*, then node *x* can be fathomed. Clearly, as mentioned earlier, a node can also be fathomed when its upper bound is lower than or equal to the current lower bound or when its solution is resource feasible. Of course, in the last case, the lower bound has to be updated first.

For a detailed description of the algorithm with an example, we refer to the research report of Vanhoucke et al. (1999).

6. Computational Experience

Both the exact recursive search algorithm for the max-*npv* problem and the branch-and-bound algorithm for the RCPSPDC have been coded in Visual C++, Version 4.0 under Windows NT 4.0 on a Dell personal computer (Pentium 550 MHz processor).

Demeulemeester et al. (1996) have validated the recursive search procedure for the max-*npv* problem against Grinold's procedure (1972). Therefore, both procedures have been coded by the same person (which makes the results very comparable) and have been validated on two data sets. The authors have shown that the recursive procedure is, on the average, 2.5 times faster than the one by Grinold. In addition, Vanhoucke et al. (1999) have shown that max-*npv* problems can be solved within a very small amount of CPU time (on the average 1.577 milliseconds) allowing for an efficient upper-bound calculation of the RCPSPDC.

To validate the branch-and-bound algorithm for the RCPSPDC, we used two different problem sets. The first contains the problem instances from the well-known Patterson problem set (Patterson 1984) and is used to compare our results with the ones found by Icmeli and Erengüç (1996). The second problem set consists of 47,520 problem instances generated by *ProGen/Max* (Schwindt 1995).

The parameter settings of the instances of the different problem sets in activity-on-the-node format are described in the subsequent sections. All problem sets were extended with cash flows, a discount rate α , and a project deadline δ_n . The project deadlines for the instances were generated as follows. After solving the different test instances with the procedure of Demeulemeester and Herroelen (1997) to obtain the minimal makespan, we generated the project deadline δ_n by adding a constant to the minimal makespan as described in §§6.1 and 6.2.

6.1. Problem Set 1: Computational Experience on the Patterson Problem Set

Icmeli and Erengüç (1996) have assembled a problem set containing 90 instances to run on an IBM 3090 computer with vector processor. Fifty of the test problems came from the set that was assembled by Patterson (1984), while the other 40 problems were generated by *ProGen* (Kolisch et al. 1995). Each problem instance was extended with a deadline and a randomly generated cash flow from a uniform distribution over the interval $[-5000; 10,000]$, which means that each activity has a chance of 1/3 to have a negative cash flow. When the deadlines of the problem instances were set to three time units above the minimal resource-constrained makespan, 36 of the 50 instances from the Patterson set could be solved to optimality within a time limit of 600 seconds and a maximum of 4,000 subproblems. This number of problems solved to optimality decreased when the project deadline was set, on the average, 14 units above the minimal makespan: 17 of the 50 Patterson instances could be solved to optimality within the limits of 600 seconds (no further limit on the number of subproblems was imposed). The average resource-constrained makespan of the problems in this problem set is 31.44 periods.

Table 1 displays the average number of nodes and the computational time for the 50 problems from the Patterson problem set with a cash flow distribution as described in Icmeli and Erengüç (1996). The row "SDR" denotes data with the use of the DR&H subset dominance rule, while the row "no SDR" represents the results without the use of the dominance rule.

The table reveals that the DR&H subset dominance rule decreases the total CPU time approximately 16% (from 11.67 seconds to 9.77 seconds), while the time needed to calculate the dominance rule only accounts for 7.47% of the total CPU time. Second, it appears that the branch-and-bound procedure can handle more than 10,000 solution nodes per second. Note that the number of created nodes denotes the number of times an upper bound is calculated (denoted by "the number of subproblems solved" in Icmeli and Erengüç 1996), while the number of branched nodes denotes the number of created nodes for which the upper bound is higher than the

Table 1 The Average CPU-Time and the Average Number of Nodes With and Without the Use of the *DR&H* Subset Dominance Rule for the Patterson Problem Set

	CPU time* B&B (1)	CPU time* max-npv (2)	CPU time* SDR (3)	Total CPU time* (1) + (2) + (3)	Number of created nodes	Number of branched nodes
<i>SDR</i>	0.78 7.98%	8.26 84.55%	0.73 7.47%	9.77	102,021.39	16,912.07
<i>No SDR</i>	0.81 6.94%	10.86 93.06%	0.00 0.00%	11.67	152,060.09	20,308.54

*Dell 550 MHz CPU time, in seconds; 64 MB memory.

currently best found solution. Notice that Icmeli and Erengüç (1996) have solved their test set to optimality within a time limit of 600 seconds and a maximum of 4,000 subproblems. Although we have used another computer, it appears that our procedure outperforms the one by Icmeli and Erengüç. Forty (*no SDR*) and 44 (*SDR*) of the 50 problems with a deadline set to three time units above the minimal makespan can be solved within a time limit of 600 seconds and 4,000 subproblems. Moreover, all 50 problem instances can be solved to optimality in less than 600 seconds when the deadline is set to 14 time units above the minimal makespan. Finally, the results reveal that most of the CPU time is spent on the calculation of the upper bound. This results from the fact that the max-npv problem calculates the discounted cash flow and consequently uses floating points, while the time spent in the B&B (branch-and-bound procedure) works with integer values. However, Vanhoucke et al. (1999) have shown that max-npv problems with 120 activities need

1.5 milliseconds of computation time for the calculation of the upper bound.

6.2. Problem Set 2: Computational Experience for the RCPSPDC

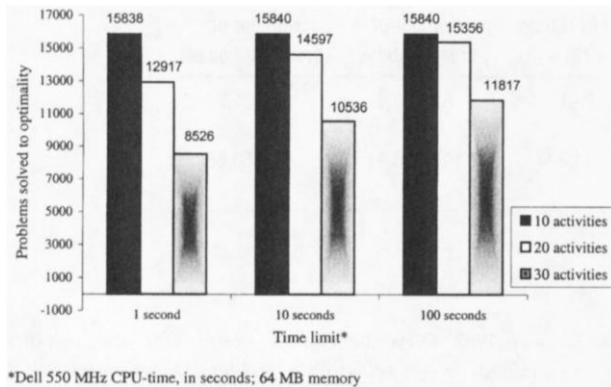
Table 2 represents the parameter settings used to generate the test instances for the second problem set for the RCPSPDC. The parameters used in the full factorial experiment are indicated in bold. We obtained 4,752 problem classes, each consisting of 10 instances. The average resource-constrained makespan of the problems in this set is 70 periods, which means that, on the average, the increase in the makespan amounts to 0% ($k = 0$), 4.29% ($k = 3$), 8.57% ($k = 6$), and 12.86% ($k = 9$).

Figure 1 displays the number of problems (out of 15,840) solved to optimality for a different number of activities and allowed CPU time. Clearly, the number of activities has a significant effect on the average CPU time and on the number of problems solved

Table 2 Parameter Settings Used to Generate the Test Instances for the RCPSPDC

Number of activities	10, 20, or 30
Activity durations	randomly selected from the interval [1,10]
Number of initial and terminal activities	randomly selected from the interval [2,4]
Maximal number of successors and predecessors	4
Order strength OS (Mastor 1970)	0.25, 0.50, or 0.75
Number of resource types	4
Number of resource types used per activity	randomly selected from the interval [1,4]
Activity resource demand	randomly selected from the interval [1,10]
Resource factor RF (Pascoe 1966)	0.25, 0.50, 0.75, or 1.00
Resource strength RS (Kolish et al. 1995)	0.00, 0.25, or 0.50
Discount rate α (in %)	0.01
Percentage negative cash flows	0%, 10%, 20%, 30%, 40%, 50%, 60%, 70%, 80%, 90%, or 100%
Deadline of the project δ_n	makespan + k with $k = 0, 3, 6$, or 9

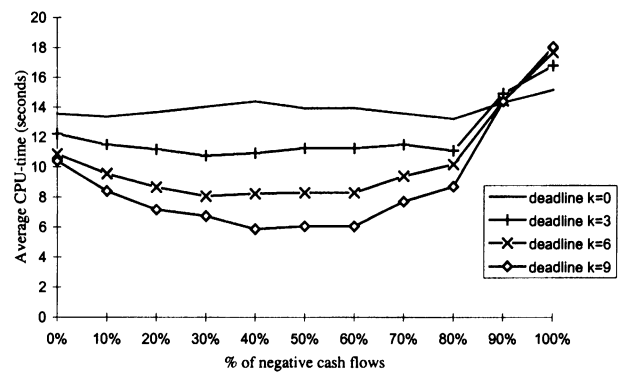
Figure 1 Effect of the Number of the Activities in a Network on the Number of Problems Solved to Optimality



to optimality. Almost all problems with 10 activities can be solved to optimality within 1 second of CPU time. For problems containing 20 activities, 81.5% of the problems can be solved to optimality when the allowed CPU time is 1 second, whereas 96.9% of the problems can be optimally solved when the time limit is 100 seconds (with an average CPU time of 5.046 seconds). For problems with 30 activities, 53.8% of the problems can be solved within 1 second of CPU time, whereas 74.6% of the problems can be solved to optimality when the allowed CPU time is 100 seconds (with an average CPU time of 29.035 seconds).

Figure 2 illustrates the impact of the cash flow distribution on the average CPU time with different project deadlines. It appears that when the project deadline is high enough, instances with few or many activities with a negative cash flow are the most difficult to solve. This is due to the fact that in these cases resource conflicts will occur very often. When there are activities both with positive and negative cash flows, the chance that a resource conflict occurs diminishes because the problem now contains activities scheduled at the beginning of the project as well as activities scheduled near the deadline. The figure also reveals that problems with a deadline equal to the resource-constrained makespan have an almost constant average CPU time. This is due to the fact that many added precedence constraints to resolve resource conflicts will lead to an infeasible solution (because the project duration exceeds the deadline) independent of the percentage of negative cash flows.

Figure 2 Effect of the % of Negative Cash Flows for Different Project Deadlines



The effects of the *OS*, the *RF*, and the *RS* seem to be in line with the literature, as indicated shortly in Vanhoucke et al. (1999).

Table 3, which displays the average number of nodes and the computational time for 7,920 problem instances with 20 activities (in fact, we have selected the instances from Table 1 with 20 activities and $k = 3$ or $k = 6$), reveals some interesting results. The table reveals that the use of the *DR&H* subset dominance rule allows for a reduction of the total CPU time with approximately 29% (from 4.09 seconds to 2.91 seconds), while the time needed to calculate the dominance rule accounts for only 13.07% of the total CPU time. Second, it appears that the branch-and-bound procedure can handle more than 20,000 solution nodes per second. Finally, as was the case for Table 1, the results also show that most of the CPU time is spent on the calculation of the upper bound.

7. Conclusions

In this paper we presented a branch-and-bound solution procedure for the resource-constrained project-scheduling problem with discounted cash flows (*RCPSDC*; $m, 1|cpm, \delta_n, c_j|npv$) based on a fast recursive search algorithm for the *max-npv* problem ($cpm, \delta_n, c_j|npv$). Each activity of this resource-constrained project-scheduling problem has a known deterministic cash flow and constant resource requirements for each renewable resource type. The objective is to schedule the activities to maximize the net

Table 3 The Average CPU-Time and the Average Number of Solution Nodes (for 20 Activity Networks with $k = 3$ or 6) With and Without the Use of the *DR&H* Subset Dominance Rule

	CPU time* B&B (1)	CPU time* max- <i>npv</i> (2)	CPU time* SDR (3)	Total CPU time* (1) + (2) + (3)	Number of created nodes	Number of branched nodes
<i>SDR</i>	0.41 14.08%	2.12 72.85%	0.38 13.07%	2.91	68,489.42	26,921.67
<i>No SDR</i>	0.69 16.87%	3.40 83.13%	0.00 0.00%	4.09	107,398.41	36,886.68

*Dell 550 MHz CPU time, in seconds; 64 MB memory.

present value of the project without violating the availabilities of the resources and subject to a project deadline. To the best of our knowledge, the procedure by Icmeli and Erengüç (1996) is the best exact procedure currently available for this problem type.

The depth-first branch-and-bound algorithm uses a new fast recursive search algorithm for the max-*npv* problem to compute upper bounds, while resource conflicts are resolved through the introduction of additional precedence constraints as described by Icmeli and Erengüç (1996). The branching strategy was extended with the subset dominance rule (De Reyck and Herroelen 1998) to prune the search tree considerably.

Both the recursive procedure for the max-*npv* problem and the branch-and-bound procedure have been coded in Visual C++, Version 4.0 under Windows NT, and have been tested on randomly generated problem sets. The results of extensive computational tests obtained on a Dell personal computer (Pentium 550 MHz) are encouraging. Unconstrained max-*npv* problems can be solved within a very small amount of CPU time, allowing for an efficient upper bound calculation. The promising results indicate that the branch-and-bound procedure is able to optimally solve instances with up to 30 activities and four resource types in a reasonable time limit and currently appears to be the best procedure available.

References

Demeulemeester, E., W. Herroelen. 1992. A branch-and-bound procedure for the multiple resource-constrained project scheduling problem. *Management Sci.* 38 1803–1818.

- , ———. 1997. New benchmark results for the resource-constrained project scheduling problem. *Management Sci.* 43 1485–1492.
- , ———, P. Van Dommelen. 1996. An optimal recursive search procedure for the deterministic unconstrained max-*npv* project scheduling problem. Research Report 9603, Department of Applied Economics, Katholieke Universiteit Leuven, Leuven, Belgium.
- De Reyck, B., W. Herroelen. 1998. A branch-and-bound procedure for the resource-constrained project scheduling problem with generalized precedence relations. *Euro. J. Oper. Res.* 111 125–174.
- Elmaghraby, S. E., ———. 1990. The scheduling of activities to maximize the net present value of projects. *Euro. J. Oper. Res.* 49 35–49.
- Etgar, R., A. Shtub. 1999. Scheduling project activities to maximize the net present value—The case of linear time dependent, contingent cash flows. *Internat. J. Production Res.* 37 329–339.
- , ———, L. J. LeBlanc. 1996. Scheduling projects to maximize net present value—The case of time-dependent, contingent cash flows. *Euro. J. Oper. Res.* 96 90–96.
- Grinold, R. C. 1972. The payment scheduling problem. *Naval Res. Logistics Quart.* 19 123–136.
- Herroelen, W., E. Demeulemeester, B. De Reyck. 1999. A classification scheme for project scheduling problems. J. Weglarz, ed. *Handbook on Recent Advances in Project Scheduling*. Kluwer Academic Publishers, Dordrecht, The Netherlands, 1–26.
- , ———, P. Van Dommelen. 1997. Project network models with discounted cash flows: A guided tour through recent developments. *Euro. J. Oper. Res.* 100 97–121.
- , E. Gallens. 1993. Computational experience with an optimal procedure for the scheduling of activities to maximize the net present value of projects. *Euro. J. Oper. Res.* 65 274–277.
- Icmeli, O., S. S. Erengüç. 1996. A branch-and-bound procedure for the resource-constrained project scheduling problem with discounted cash flows. *Management Sci.* 42 1395–1408.
- Kazaz, B., C. Sepil. 1996. Project scheduling with discounted cash flows and progress payments. *J. Oper. Res. Society* 47 1262–1272.

- Kolisch, R., A. Sprecher, A. Drexler. 1995. Characterization and generation of a general class of resource-constrained project scheduling problems. *Management Sci.* **41** 1693–1703.
- Mastor, A. A. 1970. An experimental and comparative evaluation of production line balancing techniques. *Management Sci.* **16** 728–746.
- Özdamar, L., G. Ulusoy. 1995. A survey on the resource-constrained project scheduling problem. *IIE Trans.* **27** 574–586.
- Pascoe, T. L. 1966. Allocation of resources—CPM. *Revue Française de Recherche Opérationnelle* **38** 31–38.
- Patterson, J. H. 1984. A comparison of exact procedures for solving the multiple-constrained resource project scheduling problem. *Management Sci.* **20** 767–784.
- Russell, A. H. 1970. Cash flows in networks. *Management Sci.* **16** 357–373.
- Schwindt, C. 1995. A new problem generator for different resource-constrained project scheduling problems with minimal and maximal time lags. WIOR-Report-449, Institut für Wirtschaftstheorie und Operations Research, University of Karlsruhe, Karlsruhe, Germany.
- Shtub, A., R. Etgar. 1997. A branch-and-bound algorithm for scheduling projects to maximize net present value: The case of time dependent, contingent cash flows. *Internat. J. Production Res.* **35** 3367–3378.
- Vanhoucke, M., E. Demeulemeester, W. Herroelen. 1999. On maximizing the net present value of a project under resource constraints. Research Report 9915, Department of Applied Economics, Katholieke Universiteit Leuven, Leuven, Belgium.
- Yang, K. K., F. B. Talbot, J. H. Patterson. 1992. Scheduling a project to maximize its net present value: An integer programming approach. *Euro. J. Oper. Res.* **64** 188–198.

Accepted by Luk Van Wassenhove; received March 1, 2000. This paper was with the authors 2 months for 2 revisions.